

AI Inventory Risk & Replenishment Assistant

1. Project Goal

This project is a web application for helping a company monitor inventory risks and make better replenishment decisions.

The company has:

- One warehouse
- Many products
- Historical sales data
- Different demand levels
- Different supplier lead times

The main problem is that fixed inventory rules may not work well when demand and lead time change.

The system tries to identify:

- Stockout risk
- Overstock risk
- Dead stock
- Replenishment needs
- Promotion opportunities for dead stock

It also uses AI to forecast demand and explain inventory risks.

2. My Main Goal

The main goal of this project is not only to build the application.

I wanted to practice:

1. Understanding a real business domain.
2. Designing software architecture.
3. Writing good prompts for Vibe Coding.
4. Reviewing AI-generated code.
5. Modifying an existing codebase.
6. Running and testing the application locally.
7. Understanding the code well enough to maintain it later.

The AI was used as a development assistant, not as a replacement for understanding the project.

3. Development Approach

The project was built using Vibe Coding.

My workflow was:

```
graph TD; A[Understand the business problem] --> B[Define the features]; B --> C[Think about the architecture]; C --> D[Choose the technologies]; D --> E[Write a detailed prompt]; E --> F[Let AI generate the project]; F --> G[Review the generated architecture]; G --> H[Review the code]; H --> I[Modify parts of the code]; I --> J[Run the project locally]; J --> K[Test the result];
```

The important part was that I did not simply accept the generated code without review.

4. Planned Technology Stack

The original technical plan was:

- Frontend: React
- Backend: Python + FastAPI
- Database: MySQL
- ORM: SQLAlchemy
- Migrations: Alembic (tool that helps developers track and apply changes to a database schema over time).
- Repository: One repository
- Architecture: Modular Monolith

The project was intentionally designed as an MVP (Prototype).

I did not want:

- Microservices
- Over-engineering
- A huge enterprise architecture
- Multiple warehouses

The first version focuses on one warehouse with many products.

5. What the AI Actually Built

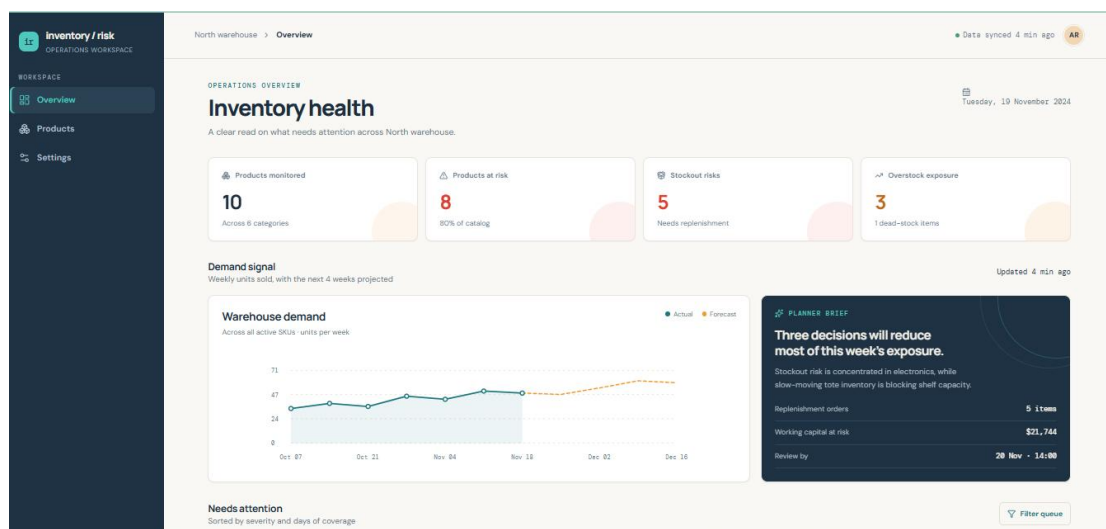
The AI generated a different technology stack from the original plan.

The current project is a:

Monorepo using pnpm workspaces

pnpm is a fast and disk-efficient package manager for Node.js projects.

Screen and run



RUN:

To run the TypeScript check:

```
pnpm --filter @workspace/inventory-risk-assistant run typecheck
```

To create a production build:

```
$env:PORT="4173"
```

```
$env:BASE_PATH="/"
```

Install

```
Npm install pnpm  
pnpm install // at project folder
```

Run:

```
pnpm --filter @workspace/inventory-risk-assistant run build /* to build for only project */  
pnpm build /*to build from package.json, use anyone*/  
pnpm exec vite preview /*final run preview*/
```

Then open:

<http://localhost:4173/>

Frontend

- React 19
- TypeScript
- Vite
- **Wouter** → A small and simple **routing library** for React. It helps you move between pages.
- **TanStack React Query** → A library for **fetching, caching, and managing server data** in React.
- **Radix UI** → A library of **accessible and customizable UI components** for React.
- **Tailwind CSS** → A CSS framework that lets you **style websites using utility classes**.
- **Lucide Icons** → A library of **open-source icons** that you can use in your application

Backend

- Express 5
- TypeScript
- CORS
- **Pino Logging** → A fast **logging library** for Node.js. It records information about what the application is doing, errors, and events.
- **Zod** → A library for **validating and checking data** in JavaScript/TypeScript.

Database

- PostgreSQL
- **Drizzle ORM** → A lightweight **ORM** that lets you work with databases using TypeScript/JavaScript instead of writing SQL for everything.
- **drizzle-zod** → A tool that **connects Drizzle schemas with Zod**, allowing you to generate Zod validation schemas from your database schemas.

API

- OpenAPI
- Generated API client

So the biggest difference is:

Planned:
React → FastAPI → MySQL/XAMPP

Actual:
React → Express → PostgreSQL

6. Current Frontend

The main frontend entry point is:

`App.tsx`

The application currently has these pages:

```
/
    Dashboard

/products
    Products list

/products/:id
    Product details

/settings
    Inventory settings
```

The UI uses React components and modern frontend libraries.

However, an important limitation exists:

The current products displayed in the frontend are still stored as static data inside `App.tsx`.

For example:

```
const products: Product[] = [...]
```

This means the frontend is currently **not fully connected to the database for product data**.

7. Current Business Logic

Some inventory classification logic already exists in the frontend.

Important functions include:

```
classify(product, settings)
```

```
severityFor(risk, product)
```

For example, Dead Stock can be detected using:

```
daysSinceLastSale >= deadStockDays
```

This means some business logic currently exists in the frontend.

A future improvement would be moving important business rules to the backend so that the backend becomes the single source of truth.

8. Current Backend

The backend uses:

Express 5 + TypeScript

The API currently has a basic health endpoint:

```
GET /api/healthz
```

It returns:

```
{
  "status": "ok"
}
```

The routes are organized through a routes index.

The backend currently does not expose the complete inventory functionality through APIs.

9. Database

The current project uses:

```
PostgreSQL
+
Drizzle ORM
```

The database connection comes from:

```
DATABASE_URL
```

The database schema is currently still very limited.

There are not yet complete Product and Inventory tables.

So the database integration is currently incomplete compared with the final business goal.

10. Main Inventory Features

The project concept contains the following features.

Demand Forecasting

The system uses historical sales data to predict future demand.

However, the AI-generated implementation only produced constant values or a very basic prediction.

My Modification

I modified the forecasting logic to use a **Moving Average**, where the prediction is calculated as the average of the last N sales values.

This makes the prediction depend on recent sales data instead of simply returning a constant value.

Stockout Risk

The system estimates whether current inventory may be insufficient.

Main idea:

$$\begin{aligned} &\text{Demand During Lead Time} \\ &= \\ &\text{Average Daily Demand} \times \text{Lead Time} \end{aligned}$$

Then:

$$\begin{aligned} &\text{Reorder Point} \\ &= \\ &\text{Demand During Lead Time} + \text{Safety Stock} \end{aligned}$$

If:

Current Stock < Reorder Point

the product may have Stockout Risk.

Overstock Risk

The system can calculate:

Days of Supply
=
Current Stock / Average Daily Demand

A very high Days of Supply can indicate Overstock Risk.

Dead Stock

A product may be considered Dead Stock when it has not sold for a long period.

Main metric:

Days Since Last Sale

Replenishment

The system can recommend when and how much to order.

The concept uses:

- Reorder Point
- EOQ

The goal is to avoid both:

Too little inventory
↓
Stockout

and:

Too much inventory
↓
Overstock

Promotion Recommendation

If a product is Dead Stock, the system can recommend a promotion.

This is mainly a business rule, not necessarily a Machine Learning model.

Explain This Risk

My planning was to make the project explain the risk using LLM.

However,

Output was just Words explaining numbers.

11. My Role vs AI Role

The AI did most of the implementation.

My role was mainly:

- Understanding the requirements.
- Designing and reviewing the architecture.
- Choosing/reviewing technology decisions.
- Writing and refining the prompts.
- Reviewing the generated code.
- Identifying differences between the planned architecture and the generated architecture.
- Modifying parts of the code.
- Running the project locally.
- Checking whether the application actually works.

So the project is better described as:

AI-assisted development with human architecture and code review.

Not:

1. **AI built everything and I did nothing.**